MS-MECANICA · BRANCH FEAT/PROJETO-INICIAL

Mapa da API da Oficina

Estrutura atual do projeto: módulos, camadas, como uma requisição atravessa o sistema e todos os endpoints do contrato com os perfis que podem chamá-los.

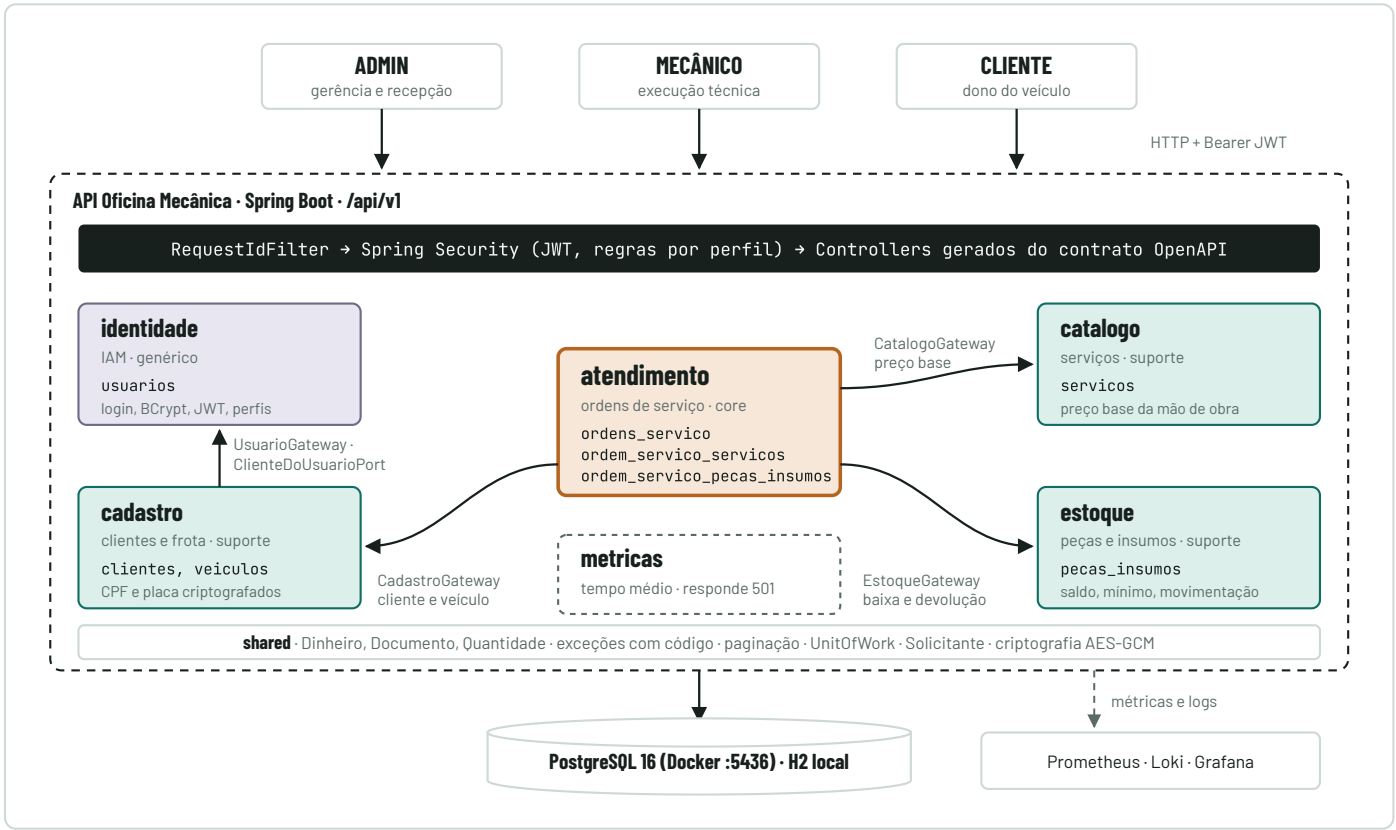
Java 21 · Spring Boot 4.1 **7** módulos **153** classes Java **38** endpoints em `/api/v1`

8 tabelas · Flyway V1–V6 Contrato OpenAPI **v4.1.0**

VISÃO GERAL

Módulos e dependências

Monólito modular: cada contexto delimitado do DDD é um pacote. As setas mostram quem depende de quem. Um módulo nunca acessa as tabelas de outro; ele chama os casos de uso do outro por meio de um gateway.



core (coração do negócio) suporte genérico não implementado

DENTRO DE CADA MÓDULO

Quatro camadas, dependências apontando para o domínio

config → adapter → application → domain. O domínio é Java puro: não conhece Spring, JPA nem HTTP.

config

Monta os casos de uso como beans e entrega as implementações dos ports.

- `AtendimentoConfig`
- `AdminInicialRunner`

adapter

Tudo que fala com o mundo de fora: HTTP, banco, outros módulos, segurança.

- `web/OrdemServicoController`
- `persistence/…JpaEntity`
- `estoque/EstoqueGatewayAdapter`
- `security/SecurityConfig`

application

Um caso de uso por operação do contrato; ports são interfaces que os adapters implementam.

- `usecase/AbrirOrdemServicoUseCase`
- `port/OrdemServicoRepository`
- `dto/OrdemServicoOutput`

domain

Entidades, agregados e value objects com as regras de negócio.

- `OrdemServico` (agregado)
- `StatusOrdemServico`
- `ItemPeca`, `ItemServico`

PASTAS

src/main/java/br/com/oficina

```
identidade/   28 classes · usuários, login, JWT
cadastro/     36 · clientes e veículos
catalogo/     14 · serviços
estoque/      17 · peças e insumos
atendimento/  34 · ordens de serviço
├── domain/
├── application/
│   ├── dto/  port/  usecase/
├── adapter/
│   ├── web/  persistence/
│   └── cadastro/  catalogo/  estoque/  ← gateways
└── config/
metricas/     1 · controller (501)
shared/       22 · núcleo compartilhado
contract/      gerado do OpenAPI a cada build
```

EXEMPLO

O cliente consulta a própria OS

1	**RequestIdFilter** gera o `X-Request-Id` e registra a chamada no log.
2	**Spring Security** valida o JWT; a rota `GET /ordens-servico/{osId}` aceita ADMIN, MECÂNICO e CLIENTE.
3	**OrdemServicoController** (implementa a interface gerada) monta o `Solicitante` a partir do token.
4	**ObterOrdemServicoUseCase** busca a OS pelo port e confere se ela é do cliente; se não for, responde 404.
5	**OrdemServicoRepositoryAdapter** lê `ordens_servico` e os itens via JPA e devolve o agregado.
6	Erros viram `application/problem+json` com `codigo` e `traceId` no **ApiExceptionHandler**.

CONTRATO OPENAPI V4.1.0

38 endpoints

Todos começam com `/api/v1`. "Só os seus" indica que o CLIENTE vê apenas os próprios registros; os de outro cliente respondem 404.

`identidade` Identidade e acessos 5

MÉTODO	ROTA	O QUE FAZ	QUEM PODE
POST	/auth/login	Autentica usuário	público
GET	/usuarios	Lista usuários	ADMIN
POST	/usuarios	Cadastra usuário interno	ADMIN
GET	/usuarios/me	Dados do usuário autenticado	ADMIN · MECÂNICO · CLIENTE
GET	/usuarios/{usuarioId}	Detalha usuário	ADMIN

`cadastro` Clientes e veículos 11

MÉTODO	ROTA	O QUE FAZ	QUEM PODE
GET	/clientes	Lista clientes	ADMIN · MECÂNICO
POST	/clientes	Cadastra cliente	público · ADMIN e MECÂNICO no balcão
POST	/clientes/pesquisa	Localiza cliente por CPF/CNPJ	ADMIN · MECÂNICO
GET	/clientes/{clienteId}	Detalha cliente	ADMIN · MECÂNICO · CLIENTE só os seus
PUT	/clientes/{clienteId}	Atualiza dados de contato do cliente	ADMIN · MECÂNICO · CLIENTE só os seus
DELETE	/clientes/{clienteId}	Remove cliente	ADMIN
GET	/veiculos	Lista veículos	ADMIN · MECÂNICO · CLIENTE só os seus
POST	/veiculos	Cadastra veículo	ADMIN · MECÂNICO · CLIENTE só os seus
GET	/veiculos/{veiculoId}	Detalha veículo	ADMIN · MECÂNICO · CLIENTE só os seus
PUT	/veiculos/{veiculoId}	Atualiza veículo	ADMIN · MECÂNICO · CLIENTE só os seus
DELETE	/veiculos/{veiculoId}	Remove veículo	ADMIN

`catalogo` Catálogo de serviços 5

MÉTODO	ROTA	O QUE FAZ	QUEM PODE
GET	/servicos	Lista serviços do catálogo	ADMIN MECÂNICO CLIENTE
POST	/servicos	Cadastra serviço	ADMIN
GET	/servicos/{servicoId}	Detalha serviço	ADMIN MECÂNICO CLIENTE
PUT	/servicos/{servicoId}	Atualiza serviço	ADMIN
DELETE	/servicos/{servicoId}	Remove serviço	ADMIN

`estoque` Peças e estoque 6

MÉTODO	ROTA	O QUE FAZ	QUEM PODE
GET	/pecas	Lista peças e insumos	ADMIN MECÂNICO
POST	/pecas	Cadastra peça ou insumo	ADMIN MECÂNICO
GET	/pecas/{pecaId}	Detalha peça/insumo	ADMIN MECÂNICO
PUT	/pecas/{pecaId}	Atualiza dados cadastrais da peça	ADMIN MECÂNICO
DELETE	/pecas/{pecaId}	Remove peça/insumo	ADMIN
POST	/pecas/{pecaId}/movimentacoes	Registra movimentação manual de estoque	ADMIN MECÂNICO

`atendimento` Ordens de serviço 10

MÉTODO	ROTA	O QUE FAZ	QUEM PODE
GET	`/ordens-servico`	Lista ordens de serviço	ADMIN · MECÂNICO · CLIENTE só os seus
POST	`/ordens-servico`	Abre ordem de serviço	ADMIN · MECÂNICO
GET	`/ordens-servico/{osId}`	Detalha ordem de serviço	ADMIN · MECÂNICO · CLIENTE só os seus
POST	`/ordens-servico/{osId}/servicos`	Inclui serviço (mão de obra) na OS	ADMIN · MECÂNICO
DELETE	`/ordens-servico/{osId}/servicos/{itemId}`	Remove serviço da OS	ADMIN · MECÂNICO
POST	`/ordens-servico/{osId}/pecas`	Inclui peça ou insumo na OS	ADMIN · MECÂNICO
DELETE	`/ordens-servico/{osId}/pecas/{itemId}`	Remove peça ou insumo da OS	ADMIN · MECÂNICO
POST	`/ordens-servico/{osId}/diagnostico`	Registra diagnóstico técnico e inicia o atendimento	ADMIN · MECÂNICO
POST	`/ordens-servico/{osId}/finalizacao`	Conclui o serviço	ADMIN · MECÂNICO
POST	`/ordens-servico/{osId}/entrega`	Registra entrega do veículo	ADMIN · MECÂNICO

`metricas` Métricas 1

MÉTODO	ROTA	O QUE FAZ	QUEM PODE
GET	`/metricas/tempo-medio-execucao`	Tempo médio de execução das OS 501 · pendente	ADMIN

Sem token, as rotas protegidas respondem `401 NAO_AUTENTICADO`; perfil sem permissão recebe `403 ACESSO_NEGADO`. Também ficam abertos `/actuator/health`, `/actuator/prometheus` e o arquivo do contrato.

Gerado a partir de `oficina-mecanica-contract.yaml` e dos pacotes em `src/main/java`.